

COMP0264 Generative AI

Exam Revision Notes: Large Language Models

Auto-generated from Lecture Slides & Recordings

UCL 2025–26

This document covers all LLM content from Lectures on 09/02, 10/02, 23/02, 24/02, and 02/03, plus slide decks `GenAI2526_LLM.pdf` and `GenAI2526_LLMs_SecondPart.pdf`.

Contents

1	Overview	2
2	Word Embedding and Tokenisation	2
2.1	The Problem with One-Hot Encoding	2
2.2	Tokenisation	3
2.3	Byte Pair Encoding (BPE)	3
3	Classical Language Models	3
3.1	Bag-of-Words Model	3
3.2	Autoregressive Models and N-grams	4
4	Recurrent Neural Networks (RNNs)	5
5	Decoder Transformers: GPT Models	6
5.1	Architecture	6
5.2	Autoregressive Generation	6
5.3	Training: Self-Supervised Learning	7
5.4	Masked (Causal) Attention	7
5.5	Padding	7
5.6	GPT Model Family	8
6	Encoder Transformers: BERT	8
6.1	Fine-Tuning BERT for Classification	9
7	Sequence-to-Sequence Transformers	9
8	Large Language Models: Scale	10
9	Training and Fine-Tuning LLMs	10
9.1	Pre-Training and Transfer Learning	10
9.2	Supervised Fine-Tuning (SFT)	10
9.3	Low-Rank Adaptation (LoRA)	11
9.4	Quantisation	11

10 Alignment: RLHF, RLVR, and DPO	12
10.1 Reinforcement Learning from Human Feedback (RLHF)	12
10.2 Reinforcement Learning from Verified Rewards (RLVR)	13
10.3 Direct Preference Optimisation (DPO)	14
10.4 RLHF vs DPO	14
11 Potential Exam Questions	14
12 References	16
12.1 Slide-Listed References	16
12.2 Lecture-Mentioned Additional Resources	16

1 Overview

This module section covers the journey from classical language models to modern Large Language Models (LLMs). The key topics are:

- Word embedding and tokenisation (Byte Pair Encoding).
- Classical language models: bag-of-words, n-gram, autoregressive models.
- Recurrent Neural Networks (RNNs) and their limitations.
- Decoder Transformers: the GPT family.
- Encoder Transformers: BERT.
- Sequence-to-Sequence Transformers.
- Scaling and the “Bitter Lesson”.
- Fine-tuning: Supervised Fine-Tuning (SFT), LoRA, Quantisation.
- Alignment: RLHF, RLVR, and DPO.

LECTURE INSIGHT:

Why LLMs matter The lecturer emphasised that the capabilities we see in modern AI (translation, summarisation, text generation) would have seemed like science fiction even in 2015. The transformer architecture, combined with scale, has essentially “solved” many classical NLP tasks. Understanding *why* these models work so well remains one of the biggest open questions in the field (mentioned in Lectures 09/02 and 10/02).

2 Word Embedding and Tokenisation

2.1 The Problem with One-Hot Encoding

One-Hot Encoding – Limitations

The first challenge is to convert words into a numerical representation suitable for a neural network. A naïve approach: define a fixed dictionary of K words and use one-hot vectors of length K , where the k -th word has 1 in position k and 0 elsewhere.

Problems:

- Extremely large, sparse vectors (e.g. $K \approx 50,000$).
- No semantic relationships captured between words.
- Curse of dimensionality makes learning difficult.
- Sparse gradient updates are computationally wasteful.

STUDENT Q&A:

Why is a large one-hot input problematic? **Student answers included:** (1) Memory size – storing and processing 50,000-dimensional sparse vectors is expensive. (2) Sparse gradients – only one position is nonzero, so gradient updates are very sparse. (3) Curse of dimensionality – learning in such a high-dimensional space is inherently difficult. (4) Not informative – numerical positions have no mapping to word semantics. The lecturer confirmed all of these are valid concerns (Lecture 09/02).

2.2 Tokenisation

Tokenisation

With a fixed dictionary, we discard important aspects of language. Tokenisation is a pre-processing step that converts a string of words and punctuation into a string of **tokens**. A token can be a word, a subword, or even a single character. The goal is to find a compromise between character-level and word-level representation.

2.3 Byte Pair Encoding (BPE)

Byte Pair Encoding (BPE)

BPE is currently one of the most popular tokenisation methods:

1. Initialise the token list with individual characters.
2. Search for the most frequently occurring pair of adjacent tokens.
3. Replace that pair with a new, merged token.
4. Do not merge across word boundaries (i.e. if the second token starts with whitespace).
5. Repeat until the desired vocabulary size is reached.

The total number of tokens is fixed in advance as a **hyperparameter** – a compromise between character-level and word-level granularity.

Reference: Sennrich et al., “Neural Machine Translation of Rare Words with Subword Units”, ACL 2016.

LECTURE INSIGHT:

How to choose the vocabulary size The vocabulary size is a hyperparameter. The lecturer described it as a “budget” – a trade-off. Choosing too few tokens compresses information excessively (losing accuracy); choosing too many increases computational cost without proportional benefit. In practice, one fixes the token budget in advance and uses performance indicators (e.g. downstream task accuracy) to find the best trade-off experimentally. The lecturer noted: “It’s basically compression – you could plot the number of tokens versus some accuracy metric and at some point adding more tokens isn’t worth the cost” (Lecture 09/02).

STUDENT Q&A:

Does BPE also handle the conversion to vector representation? No. BPE only handles the *segmentation* of text into tokens. A separate embedding layer (trained as part of the model) then maps each token to a dense vector of dimension D (Lecture 09/02).

3 Classical Language Models

3.1 Bag-of-Words Model

Bag-of-Words Model

The simplest model assumes words are drawn independently from the same distribution:

$$p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n)$$

This can be represented as a simple probability table. However, it completely **ignores the order** of words.

LECTURE INSIGHT:

When bag-of-words is useful Despite its simplicity, bag-of-words can be useful for very fast, cheap statistics about text (e.g. basic search, topic detection). Frequency-based lookups in a table are extremely fast. However, for generation tasks, word order is essential (Lecture 09/02).

3.2 Autoregressive Models and N-grams**Autoregressive Decomposition**

We can decompose the joint distribution over a sequence of tokens into a product of conditional distributions:

$$p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n \mid x_1, \dots, x_{n-1})$$

Each term could in principle be estimated via frequency counts from a training corpus. However, the size of the required probability table grows **exponentially** with sequence length.

N-gram Models

To make the problem tractable, we assume each conditional distribution depends only on the L most recent tokens:

- $L = 1$: **Bigram** model – $p(x_n \mid x_{n-1})$
- $L = 2$: **Trigram** model – $p(x_n \mid x_{n-1}, x_{n-2})$
- General: **N-gram** model

For a trigram model:

$$p(x_1, \dots, x_N) = p(x_1) p(x_2 \mid x_1) \prod_{n=3}^N p(x_n \mid x_{n-1}, x_{n-2})$$

LECTURE INSIGHT:

The sheer scale of the full autoregressive table The lecturer walked through a calculation: for a dictionary of $D = 50,000$ words and a sequence of $N = 100$ tokens, the full probability table for predicting the last word requires $D^{N-1} = 50,000^{99}$ entries – vastly exceeding the number of atoms in the universe. This is why we need neural networks as function approximators to *compress* these probability distributions into manageable parameter spaces (Lecture 09/02).

EXAM ALERT:

Why neural networks for language modelling? The lecturer explicitly framed this as an exam-relevant reasoning chain: We want to model $p(x_n \mid x_1, \dots, x_{n-1})$. The full table is exponentially large $\Rightarrow$ need parametric approximation $\Rightarrow$ neural networks are universal function approximators $\Rightarrow$ they can compress these massive probability tables into learnable weight matrices (Lecture 09/02).

4 Recurrent Neural Networks (RNNs)

RNN Architecture

An RNN introduces a hidden variable z_n at each step n . The network takes as input the current token x_n and the current hidden state z_{n-1} , producing output y_n and the next hidden state z_n . Copies of the network are chained together with **shared weights** w .

Training uses stochastic gradient descent with **backpropagation through time (BPTT)**. The loss is typically the cross-entropy over all output positions, with softmax activations.

RNN for Translation (Encoder-Decoder)

For translation tasks (variable-length input $\rightarrow$ variable-length output):

- The **encoder** processes the input sentence token by token, building up a hidden state.
- A special $\langle \text{start} \rangle$ token signals the transition to decoding.
- The **decoder** generates output tokens autoregressively, feeding each generated token back as input.
- A $\langle \text{stop} \rangle$ token signals end of translation.
- Hidden state initialised to $z_0 = (0, \dots, 0)^T$.

RNN Limitations

1. **Vanishing/Exploding gradients:** For very long sequences, gradient signals decay or blow up during BPTT.
2. **Bottleneck problem:** The entire input sequence must be summarised into a single fixed-size hidden vector z , which is insufficient for long sequences.
3. **Poor parallelisation:** Sequential dependencies prevent efficient GPU computation.
4. **Poor long-range dependencies:** Information from early tokens is progressively lost.

LECTURE INSIGHT:

The bottleneck intuition The lecturer emphasised: “You are summarising everything that comes before – potentially 99 words – with *one* latent variable. That is a bottleneck.” This compression is fundamentally too lossy for rich language tasks. This bottleneck problem, combined with vanishing gradients, is exactly why attention mechanisms (and therefore transformers) were developed (Lectures 09/02 and 10/02).

LECTURE INSIGHT:

Historical note – LSTM and GRU The lecturer mentioned LSTM (Hochreiter & Schmidhuber, Neural Computation 1997) and GRU (Cho et al., EMNLP 2014) as historical improvements to vanilla RNNs. He also referenced Andrej Karpathy’s famous blog post on generating Shakespeare with RNNs (circa 2015) – a landmark moment that demonstrated the potential of neural text generation. These are now “classic” models, superseded by transformers (Lecture 09/02).

EXAM ALERT:

RNN $\rightarrow$ Transformer motivation Be prepared to explain the full reasoning chain: n-grams don’t scale $\rightarrow$ RNNs offer parametric models but suffer from vanishing gradients, bottleneck, and poor parallelisation $\rightarrow$ attention mechanisms solve long-range dependencies $\rightarrow$ transformers stack attention layers, are parallelisable, and enable modern LLMs (Lecture 09/02).

5 Decoder Transformers: GPT Models

5.1 Architecture

GPT Architecture

A GPT (Generative Pre-trained Transformer) model is a **decoder-only** transformer used as an autoregressive generative model:

$$p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n \mid x_1, \dots, x_{n-1})$$

The architecture consists of:

1. **Token embedding + positional encoding** for each input token.
2. A stack of L **masked transformer layers**.
3. A **Linear-Softmax (LSM)** output head: a linear transformation $W^{(p)} \in \mathbb{R}^{D \times K}$ followed by softmax, where K is the vocabulary size.

The output at each position is a probability distribution over the dictionary:

$$Y = \text{softmax}(\tilde{X} W^{(p)})$$

where $\tilde{X}$ is the output of the last transformer layer (dimension $N \times D$), and Y has dimension $N \times K$.

LECTURE INSIGHT:

Input/Output dimensionality walkthrough The lecturer carefully walked through the dimensions using GPT-2 (small) as an example:

- Each token is represented as a vector of dimension $D = 768$.
- A sentence of N tokens is represented as a matrix $X \in \mathbb{R}^{N \times D}$.
- The transformer layers preserve dimension: output is also $N \times D$.
- The final LSM layer transforms from $D = 768$ to $K = 50,257$ (vocabulary size in GPT-2).
- Batch processing: with batch size B and sequence length N , the tensor shape is (B, N, D) .

This is “the difficult bit” – understanding these shapes is key to understanding everything else (Lecture 10/02).

5.2 Autoregressive Generation

GPT Generation Process

Given $n - 1$ input tokens, the model outputs a probability distribution $p(x_n \mid x_1, \dots, x_{n-1})$ over the vocabulary. The process:

1. Feed tokens $[\langle \text{start} \rangle, x_1, \dots, x_{n-1}]$ into the model.
2. Obtain probability distribution over the n -th token.
3. Sample or select a token from this distribution.
4. Append the new token to the sequence and repeat.
5. Continue until an $\langle \text{end} \rangle$ token is generated.

5.3 Training: Self-Supervised Learning

Self-Supervised Training of GPT

GPT models are trained on large **unlabelled** corpora through **self-supervision**:

- Each training sample: input = $(x_1, \dots, x_n)$, target = x_{n+1} .
- The ground truth is already in the text – no human labels needed.
- The loss function is **cross-entropy** over mini-batches of sequences.
- A single sentence yields *multiple* training examples: “The cat sat on the mat” gives 5 next-token prediction tasks.

LECTURE INSIGHT:

Power of self-supervision The lecturer stressed: “This is powerful because you don’t need to pay anyone to label your data. You have massive amounts of text for free – Wikipedia, books, the internet. You simply predict the next word, and the correct answer is already there in the text.” This is why LLMs can be trained on trillions of tokens (Lecture 10/02).

5.4 Masked (Causal) Attention

Masked Attention

To train on all positions in parallel without the model “cheating” by looking ahead:

1. Shift the input sequence right by one step: input x_n corresponds to target x_{n+1} . A $\langle \text{start} \rangle$ token is prepended.
2. In the attention mechanism, set to **zero** all attention weights where token n attends to any token $m > n$ (future tokens).
3. **Implementation trick**: Set pre-softmax attention values to $-\infty$ for future positions. Since $e^{-\infty} = 0$, the softmax automatically zeros out future contributions *and* correctly normalises the remaining weights.

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}[\text{mask}(QK^T)] V$$

where $\text{mask}(\cdot)$ sets upper-triangular entries to $-\infty$.

EXAM ALERT:

The $-\infty$ trick for causal masking The lecturer devoted significant time to explaining this trick and even derived it at the board. Key insight: setting pre-softmax values to $-\infty$ achieves two things simultaneously: (1) the attention weight becomes exactly zero (since $e^{-\infty} = 0$), and (2) the normalisation in the denominator of softmax is automatically correct because the zero terms don’t contribute to the sum. This is elegant because it handles variable-length masking without any special normalisation code (Lecture 10/02).

5.5 Padding

Padding for Batch Processing

GPUs require fixed-size tensors. When sequences have different lengths:

- Introduce $\langle \text{pad} \rangle$ tokens to fill shorter sequences to the maximum length.
- An additional mask sets attention weights to zero for all pad tokens.

Note: Masking (causal) and padding are **two independent** mechanisms. Masking prevents looking at future tokens; padding handles variable-length sequences for GPU parallelism.

STUDENT Q&A:

What about repeated tokens in a sequence? A student asked: if a token appears twice (e.g. “the” in “the cat sat on **the** mat”), doesn’t masking cause problems? Answer: No. Each token position is treated as a separate element in the sequence. The second “the” at position 5 is masked from seeing “mat” at position 6, but it can attend to all previous positions (1–4) including the first “the”. Repeated tokens are perfectly valid – the model can predict that the next token is the same as a previous one (Lecture 10/02).

5.6 GPT Model Family**GPT Timeline**

- **GPT-1**: OpenAI, 2018
- **GPT-2**: OpenAI, 2020 (small version: $D = 768$, vocab size $K = 50,257$)
- **GPT-3**: OpenAI, 2023 (175B parameters)
- **GPT-4**: OpenAI, 2023 (arXiv:2303.12712 – “Sparks of AGI” paper from Microsoft)

Reference: Chapter 12 of Bishop & Bishop, *Deep Learning Foundations and Concepts*, Springer 2024.

LECTURE INSIGHT:

Scaling and emergent behaviour The lecturer expressed fascination with the *scaling* phenomenon: stacking more transformer layers and training on more data leads to emergent capabilities that appear qualitatively different. “Why does stacking really help? For one thing, more layers means more attention patterns. Each head in each layer learns some type of dependency. But exactly *why* this leads to what looks like intelligence is something we still don’t understand.” He referenced interpretability research showing that modifying individual attention layer weights can change the model’s factual outputs (e.g. changing the answer to “What is the capital of France?”) (Lecture 09/02).

6 Encoder Transformers: BERT**BERT – Bidirectional Encoder Representations from Transformers**

BERT is an **encoder-only** transformer designed for classification and other discriminative tasks. Key differences from GPT:

- Uses **bidirectional** attention (no causal mask) – each token can attend to all other tokens, both before and after.
- Pre-trained via **Masked Language Modelling (MLM)**: randomly mask 15% of tokens with $\langle \text{mask} \rangle$ and train the model to predict them.
- A special $\langle \text{class} \rangle$ token is prepended; its output is used for classification tasks.

Reference: Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, ACL 2019.

BERT Masking Strategy

To mitigate the distributional mismatch between pre-training (which uses $\langle \text{mask} \rangle$) and fine-tuning (which does not), the original BERT paper modifies the 15% selected tokens as follows:

- 80% are replaced with $\langle \text{mask} \rangle$.
- 10% are replaced with a random word.
- 10% are left unchanged.

This introduces noise that makes the model more robust during fine-tuning.

LECTURE INSIGHT:

BERT vs GPT: bidirectional vs causal The lecturer emphasised: “BERT can ‘see’ tokens both before *and* after the masked position – that’s why it’s called bidirectional. In GPT, we mask the future because we’re generating text left-to-right. In BERT, we’re not generating – we’re *understanding* – so seeing the full context is an advantage.” However, BERT is less efficient for training than GPT because only 15% of tokens contribute to the loss at each step, whereas in GPT every token is a training signal (Lecture 23/02).

6.1 Fine-Tuning BERT for Classification

BERT Fine-Tuning for Classification

For a text classification task with K classes:

- Use only the output corresponding to the $\langle \text{class} \rangle$ token (first position).
- Append a parameter matrix $W \in \mathbb{R}^{D \times K}$ to this output.
- Apply softmax (for $K > 2$) or logistic sigmoid (for $K = 2$).
- Fine-tune all parameters using SGD with cross-entropy loss.

For **token-level classification** (e.g. named entity recognition): ignore the $\langle \text{class} \rangle$ output; use subsequent outputs with a shared linear layer + softmax.

LECTURE INSIGHT:

Fine-tuning philosophy “You pre-train BERT on massive amounts of text. Then you *specialise* it for a specific task by providing classification examples. Think of the transformer layers as a powerful encoder, and the classification head as a simple task-specific decoder. This is transfer learning” (Lecture 23/02).

7 Sequence-to-Sequence Transformers

Encoder-Decoder Transformer

The original “Attention is All You Need” (2017) architecture uses both encoder and decoder:

- The **encoder** maps the input token sequence X into an internal representation Z using self-attention transformer layers.
- The **decoder** generates output tokens using masked self-attention *plus* **cross-attention** to the encoder output Z .
- In cross-attention: K and V come from the encoder output Z , while Q comes from the decoder’s current representation.
- Trained on paired input-output sequences.

Decoder-Only Translation

It is possible to use a decoder-only transformer for translation by simply formatting the input as:

Translate $\langle \text{sentence in language X} \rangle \langle \text{sentence in language Y} \rangle$

In this case, encoding happens **implicitly** within the decoder. This is the approach used by modern LLMs.

8 Large Language Models: Scale

What Makes LLMs “Large”?

LLMs are very large transformer-based models with potentially more than 10^{12} (1 trillion) parameters. Their success is enabled by:

- Availability of massive training datasets.
- GPU hardware enabling parallel computation.
- The transformer architecture’s efficient use of parallelism.

It is about scale.

The Bitter Lesson (Rich Sutton)

Reference: <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>

The “Bitter Lesson” argues that general methods that leverage computation (search and learning) tend to outperform methods that leverage human knowledge, given sufficient scale. This philosophy underpins the LLM approach: rather than encoding linguistic rules, simply scale up data and compute.

9 Training and Fine-Tuning LLMs

9.1 Pre-Training and Transfer Learning

Pre-Training and Fine-Tuning Pipeline

Models are first **pre-trained** using self-supervised learning on massive corpora. This provides general language understanding. The model is then **fine-tuned** for specific downstream tasks using smaller labelled datasets. This is **transfer learning**.

Fine-tuning approaches:

1. Adjust **all** weights and biases.
2. Add extra layers to the output and train only those.
3. Replace the last few layers with fresh parameters and train them.

During fine-tuning, the main model weights can be frozen or allowed small adaptation.

9.2 Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning (SFT)

SFT adapts a pre-trained LLM to specific tasks using human-labelled demonstrations. Given a dataset $\{(x_i, y_i)\}$ where x_i is a prompt and y_i is the desired response, the loss is:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{i=1}^N \sum_{t=1}^{T_i} \log P_{\theta}(y_{i,t} \mid y_{i,<t}, x_i)$$

where $y_{i,t}$ is the token at time step t , $y_{i,<t}$ are preceding tokens, and θ are model parameters. SFT provides a **baseline policy** for downstream alignment (e.g. RLHF) or task specialisation.

9.3 Low-Rank Adaptation (LoRA)

LoRA: Low-Rank Adaptation

LoRA exploits the observation that fine-tuning changes lie on a **low-dimensional manifold**: the intrinsic dimensionality of fine-tuning updates is much smaller than the total parameter count.

Method: Freeze the original weight matrix $W_0 \in \mathbb{R}^{D \times D}$ and add a low-rank update:

$$\hat{W} = W_0 + AB$$

where $A \in \mathbb{R}^{D \times R}$ (down-projection) and $B \in \mathbb{R}^{R \times D}$ (up-projection), with $R \ll D$.

The layer output becomes:

$$\text{output} = XW_0 + XAB$$

Key properties:

- Number of new parameters: $2RD \ll D^2$ (original matrix).
- Typically only attention-layer weights are modified; MLP weights stay fixed.
- After fine-tuning, AB can be merged into W_0 : $\hat{W} = W_0 + AB$ – no inference overhead.

Reference: Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models”, arXiv:2106.09685, 2021.

LECTURE INSIGHT:

Intuition for LoRA A is a **down-projection**: compresses the input from high-dimensional space (D) into a small bottleneck of size R . “Think of it like projecting a 3D object to a 2D plane.” B is an **up-projection**: expands back to dimension D . This is the same concept as PCA – projecting into a lower-dimensional subspace that captures the most important variation.

Why it works: The authors found that task-specific adaptation only affects a small number of directions in weight space. “The model doesn’t need to update all dimensions – just the meaningful ones. The low-rank projection captures exactly those directions” (Lecture 03/02).

LECTURE INSIGHT:

LoRA as the dominant PEFT method LoRA has become the de facto standard for **Parameter-Efficient Fine-Tuning (PEFT)**. The lecturer noted that “parameter efficient” means both fewer parameters to train *and* less computational cost, which translates to less energy and less money. This is critical when fine-tuning models with trillions of parameters (Lecture 03/02).

9.4 Quantisation

Quantisation

Modern LLMs have billions/trillions of parameters requiring hundreds of GBs in FP32. The limiting factor for inference is often **memory bandwidth**, not FLOPs.

Quantisation: mapping floating-point weights/activations to lower-precision discrete integers (INT4, INT8). Benefits:

- Significantly less memory per parameter.
- Lower-precision computations reduce energy consumption.

Floating-point representation: $x = (-1)^{\text{sign}} \times 2^{\text{exponent}} \times \text{mantissa}$

Example: FP8 E4M3 uses 4 bits for exponent, 3 for mantissa. Range: approximately $[-448, 448]$ vs FP16’s $[-65504, 65504]$.

Affine Quantisation

Affine quantisation uses two parameters:

- **Scale factor** s : determines step size.
- **Zero point** z : ensures exact zero mapping (important for zero-padding in neural networks).

Quantisation formula:

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s} + z\right), \alpha_q, \beta_q\right)$$

Dequantisation (approximate recovery):

$$x \approx x' = s(x_q - z)$$

This is inherently lossy – a trade-off between performance and precision.

QLoRA

QLoRA combines quantisation with LoRA: quantise the base model to low precision, then apply LoRA adapters in higher precision. This enables fine-tuning very large models on limited hardware.

Reference: Dettmers et al., “QLoRA: Efficient Finetuning of Quantized Language Models”, NeurIPS 2023.

LECTURE INSIGHT:

Quantisation in practice The lecturer explained the connection to GPU hardware: NVIDIA GPUs (A100, H100) support specific number formats (FP32, FP16, FP8, INT8, INT4) through CUDA. PyTorch compiles for CUDA, which maps to the actual hardware. There are many active research proposals for both **post-training quantisation** and **quantisation-aware training** (Lecture 03/02).

10 Alignment: RLHF, RLVR, and DPO

10.1 Reinforcement Learning from Human Feedback (RLHF)

RLHF: Motivation and Pipeline

SFT optimises token-level likelihood, *not* human preference directly. RLHF improves output quality by incorporating human evaluation.

Pipeline:

1. Collect **comparative human feedback**: pairs of model outputs for the same prompt, labelled with which is preferred.
2. Train a **reward model** $R_\theta(y | x)$ to predict human preferences.
3. Use the reward model to judge model outputs.
4. Apply RL (e.g. PPO or GRPO) to update LLM weights to maximise cumulative reward.

RLHF: Reward Model Training

Given comparison data (x, y_1, y_2) where $y_1 \succ y_2$ (humans prefer y_1), the reward model satisfies:

$$P(y_1 \succ y_2 | x) = \frac{\exp(R_\theta(y_1 | x))}{\exp(R_\theta(y_1 | x)) + \exp(R_\theta(y_2 | x))}$$

The loss is binary cross-entropy:

$$\mathcal{L}_{\text{RM}}(\theta) = - \sum_{(x, y_1, y_2)} \log P(y_1 \succ y_2 \mid x)$$

RLHF: Policy Optimisation

The language model is updated via RL, maximising expected reward while staying close to a reference policy π_{ref} :

$$\mathcal{L}_{\text{RL}}(\phi) = -\mathbb{E}_{y \sim \pi_{\phi}}[R_{\theta}(y \mid x)] + \beta \text{KL}(\pi_{\phi} \parallel \pi_{\text{ref}})$$

The **KL regularisation** prevents the model from diverging too far from the original SFT-trained model.

RLHF: Potential Issues

- **Reward model bias**: RM may inherit annotator subjectivity or dataset skew.
- **Exploitation risk** (reward hacking): the LLM can find loopholes in the RM, producing outputs that maximise reward but are undesirable.
- **Data bottleneck**: high-quality human feedback is expensive to collect.

Reference: Ziegler et al., “Fine-Tuning Language Models from Human Preferences”, OpenAI, 2019.

LECTURE INSIGHT:

Why use a reward model instead of humans directly? “Asking a user to label potentially billions of outputs is unfeasible. The reward model is trying to replace humans as judges. Ideally, we want humans to give every answer, but to scale up, we train a model *of* human preferences. Having a good reward model is one of the most important and expensive things a company does” (Lecture 03/02).

LECTURE INSIGHT:

KL regularisation intuition “You want to score the maximum reward by generating a ‘good’ output while being judged by a reward model. But you also penalise the model if it diverges too far from the original distribution – otherwise it might find degenerate solutions that game the reward” (Lecture 03/02).

10.2 Reinforcement Learning from Verified Rewards (RLVR)

RLVR

In RLVR, the reward is derived from **objective, verifiable criteria** rather than subjective human rankings. No humans involved in reward computation.

Examples: factual correctness checks, code compilation success, adherence to safety rules.

Pros: Scalable; less subjective bias.

Cons: May not capture nuanced human preferences or naturalness; limited by the chosen verification metric.

10.3 Direct Preference Optimisation (DPO)

DPO: Motivation and Objective

DPO is an alternative to RLHF that **directly** optimises the model from preference data, *without* a separate reward model or RL step.

Given a prompt x and two responses: y^+ (preferred) and y^- (non-preferred), DPO maximises $P_\theta(y^+ \succ y^- | x)$.

Loss function (pairwise logistic loss):

$$\mathcal{L}_{\text{DPO}}(\theta) = - \sum_{(x, y^+, y^-)} \log \sigma(\log \pi_\theta(y^+ | x) - \log \pi_\theta(y^- | x))$$

where σ is the sigmoid function and $\pi_\theta(y | x)$ is the model's probability of generating y given x .

DPO Workflow

1. Prompt x and generate candidate responses y^+, y^- .
2. Compare log-probabilities using the logistic loss.
3. Backpropagate to maximise preference for y^+ .

DPO simplifies alignment into a **single, stable optimisation objective**, unlike RLHF's two-step process (reward model training + RL).

Reference: Rafailov et al., “Direct Preference Optimisation”, NeurIPS 2023.

10.4 RLHF vs DPO

RLHF vs DPO: Comparison

Aspect	RLHF	DPO
Architecture	Two-step: reward model + RL	Single optimisation objective
Reward model	Explicit, reusable	None (implicit)
Modularity	High – RM can enforce complex objectives (safety, factuality, style)	Low – changing goals requires new preference data
Scalar signal	Yes – can be analysed/reused	No
Sparse data	RM smooths over sparse comparisons	Can overfit with sparse or noisy data
Stability	Can be unstable (RL training)	Generally more stable

11 Potential Exam Questions

EXAM ALERT:

Explain the autoregressive language model and why n-grams don't scale Explain the decomposition $p(x_1, \dots, x_N) = \prod p(x_n | x_{1:n-1})$. Show why the full probability table grows as $|V|^N$ and is therefore intractable. Explain how neural networks serve as function approximators to compress this table (mentioned repeatedly in Lecture 09/02, described as “exam-like reasoning”).

EXAM ALERT:

Compare RNNs and Transformers for language modelling Discuss: (1) vanishing/exploding gradients in RNNs, (2) the bottleneck problem (summarising entire history in one hidden vector), (3) inability to parallelise RNNs, (4) how attention solves long-range dependencies, (5) how causal masking enables parallel training in transformers (Lectures 09/02, 10/02).

EXAM ALERT:

Explain causal masking in GPT and the $-\infty$ trick Describe why masking is needed (prevent cheating by looking at future tokens during training). Explain the mechanism: set pre-softmax values to $-\infty$ so softmax outputs zero, with automatic correct normalisation. Distinguish masking (causal attention) from padding (handling variable-length sequences) (Lecture 10/02 – the lecturer spent significant time on this).

EXAM ALERT:

Compare BERT and GPT architectures BERT: encoder-only, bidirectional attention, masked language modelling (predict 15% masked tokens), used for classification. GPT: decoder-only, causal attention, next-token prediction, used for generation. Discuss training efficiency (GPT uses all tokens; BERT only 15%). Mention BERT's 80/10/10 masking strategy (Lectures 23/02, slides).

EXAM ALERT:

Explain LoRA and why it works Describe: freeze W_0 , add low-rank AB where $A \in \mathbb{R}^{D \times R}$, $B \in \mathbb{R}^{R \times D}$, $R \ll D$. Explain parameter count: $2RD \ll D^2$. Discuss the intrinsic dimensionality argument. Explain merge at inference: $\hat{W} = W_0 + AB$ with no overhead. Mention it as the dominant PEFT method (Lectures 03/02, slides).

EXAM ALERT:

Explain RLHF pipeline and the reward model Full pipeline: SFT $\rightarrow$ reward model training from human comparisons $\rightarrow$ RL optimisation with KL regularisation. Write the reward model loss (binary cross-entropy on Bradley-Terry preferences) and the RL objective with KL penalty. Discuss potential issues: reward hacking, annotator bias, data cost (Lectures 03/02, slides).

EXAM ALERT:

Compare RLHF and DPO RLHF: two-step (reward model + RL), modular, scalar signal reusable, can be unstable. DPO: single-step, directly optimises from preferences, more stable, but less modular and can overfit with sparse data. Write DPO loss: $\mathcal{L} = -\sum \log \sigma(\log \pi_{\theta}(y^+|x) - \log \pi_{\theta}(y^-|x))$ (Slides LLM Part 2).

EXAM ALERT:

Explain quantisation and affine quantisation formula Motivation: memory bandwidth is the bottleneck; INT4/INT8 use less memory. Write the affine quantisation formula: $x_q = \text{clip}(\text{round}(x/s + z), \alpha_q, \beta_q)$ and dequantisation: $x' = s(x_q - z)$. Explain scale factor and zero point. Mention QLoRA as combining quantisation with LoRA (Lectures 03/02, slides).

EXAM ALERT:

SFT loss function Write and explain: $\mathcal{L}_{\text{SFT}}(\theta) = -\sum_i \sum_t \log P_{\theta}(y_{i,t} \mid y_{i,<t}, x_i)$. This is standard cross-entropy / maximum likelihood estimation on (prompt, response) pairs. Understand that SFT optimises token-level likelihood, not human preference – which motivates RLHF/DPO (Slides LLM Part 2).

12 References

12.1 Slide-Listed References

- C. Bishop & H. Bishop, *Deep Learning Foundations and Concepts*, Springer, 2024. Chapter 12.
- Sennrich et al., “Neural Machine Translation of Rare Words with Subword Units”, ACL 2016 (BPE).
- Hochreiter & Schmidhuber, “Long Short-Term Memory”, Neural Computation, 1997 (LSTM).
- Cho et al., “Learning Phrase Representations using RNN Encoder-Decoder”, EMNLP 2014 (GRU).
- Radford et al., “Improving Language Understanding by Generative Pre-Training”, OpenAI 2018 (GPT-1).
- Radford et al., “Language Models are Unsupervised Multitask Learners”, OpenAI 2020 (GPT-2).
- Brown et al., “Language Models are Few-Shot Learners”, OpenAI 2023 (GPT-3).
- OpenAI, “GPT-4 Technical Report”, arXiv:2303.12712, 2023.
- Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers”, ACL 2019.
- Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models”, arXiv:2106.09685, 2021.
- Dettmers et al., “QLoRA”, NeurIPS 2023.
- Ziegler et al., “Fine-Tuning Language Models from Human Preferences”, OpenAI 2019 (RLHF).
- Rafailov et al., “Direct Preference Optimisation”, NeurIPS 2023 (DPO).
- Rich Sutton, “The Bitter Lesson”, <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- NVIDIA documentation on quantisation.

12.2 Lecture-Mentioned Additional Resources

- Andrej Karpathy’s blog post on generating Shakespeare with RNNs (referenced in Lecture 09/02).
- Bubeck et al., “Sparks of Artificial General Intelligence: Early Experiments with GPT-4”, Microsoft Research (arXiv:2303.12712, referenced in slides).
- HuggingFace **transformers** library – used in tutorials for hands-on GPT-2 experimentation.
- HuggingFace **trl** library – used for SFT and RL-based fine-tuning in coursework.
- HuggingFace **pft** library – used for LoRA configuration in coursework.